



POSTECH
포항공과대학교



SEOUL
NATIONAL
UNIVERSITY

StressRight: **Finding the Right Stress for** **Accurate In-development System Evaluation**

Jaewon Lee¹, Hanhwi Jang¹, Jae-eon Jo¹,
Gyu-Hyeon Lee², Jangwoo Kim²

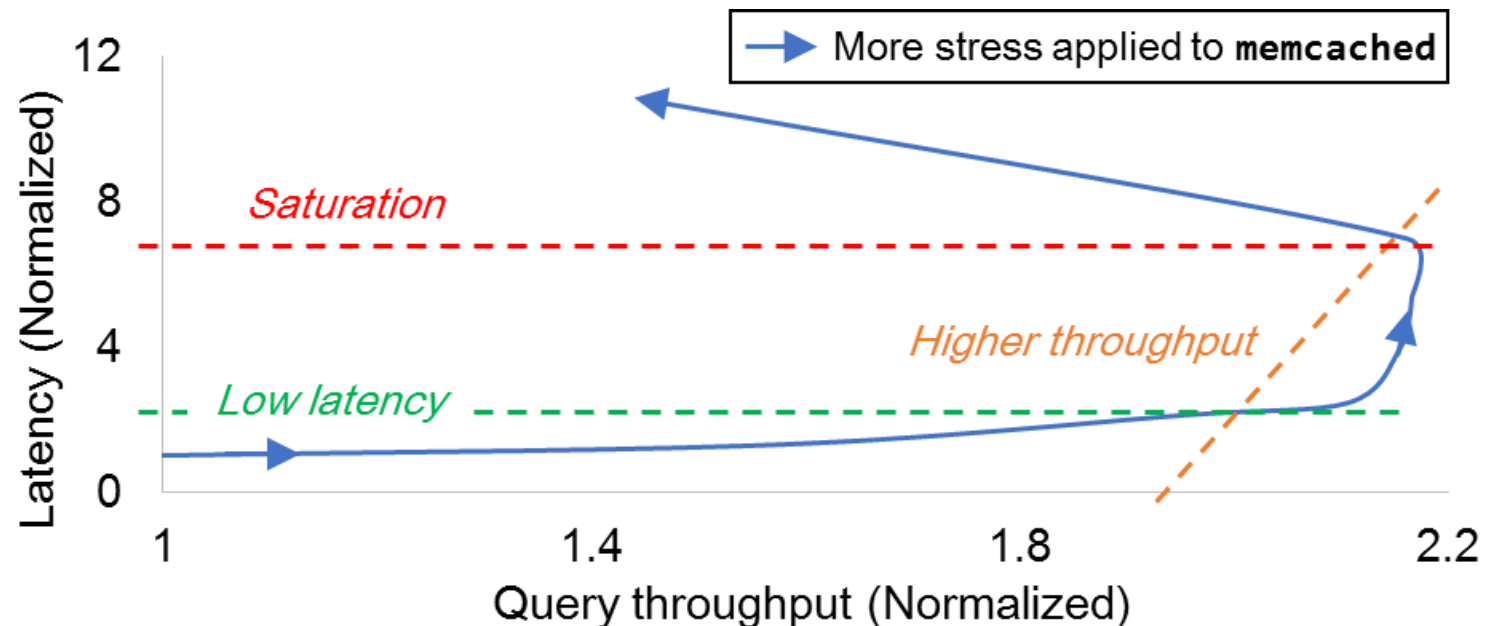
High Performance Computing Lab

Pohang University of Science and Technology (POSTECH)¹

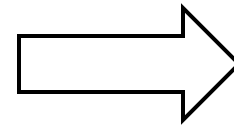
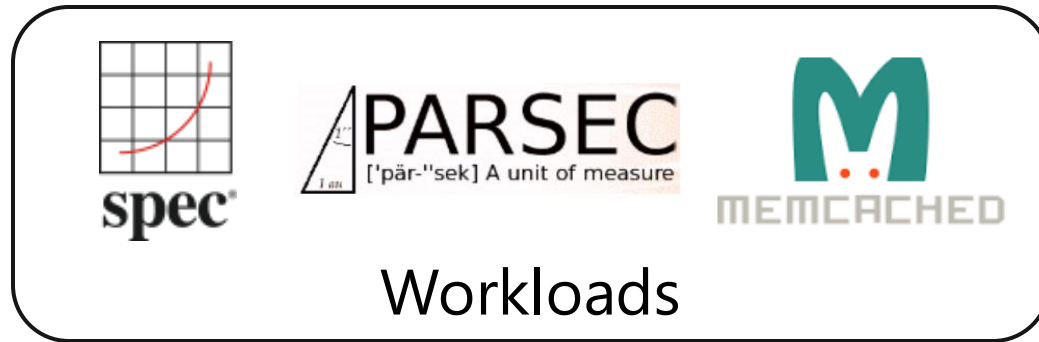
Seoul National University²

Configuring Workloads

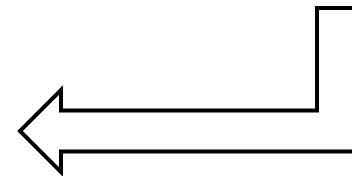
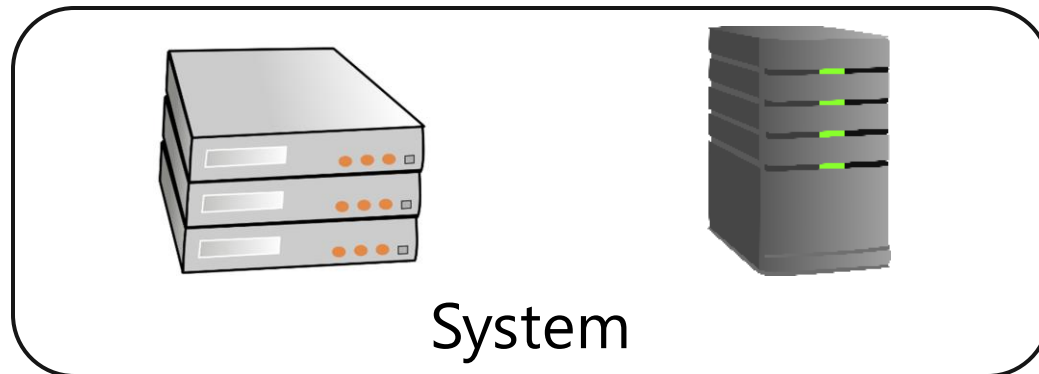
- **Modern workloads are configurable**
 - No definite answer: depends on the usage scenario



Evaluating a System

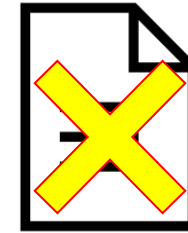
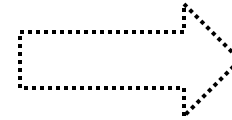
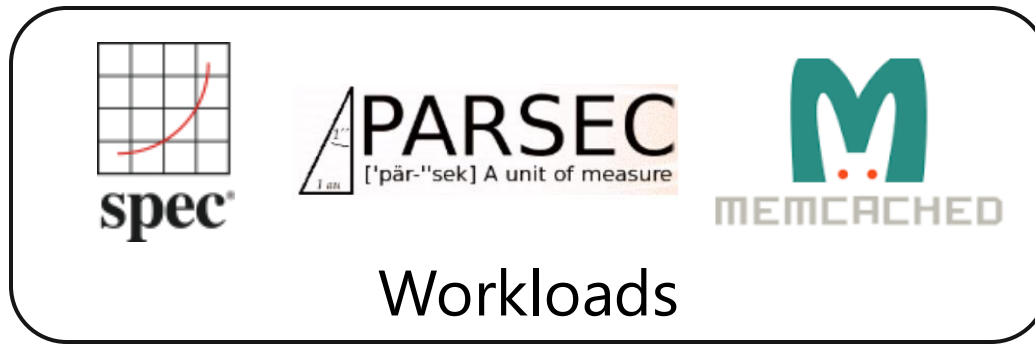


Performance report
(e.g., latency, throughput)

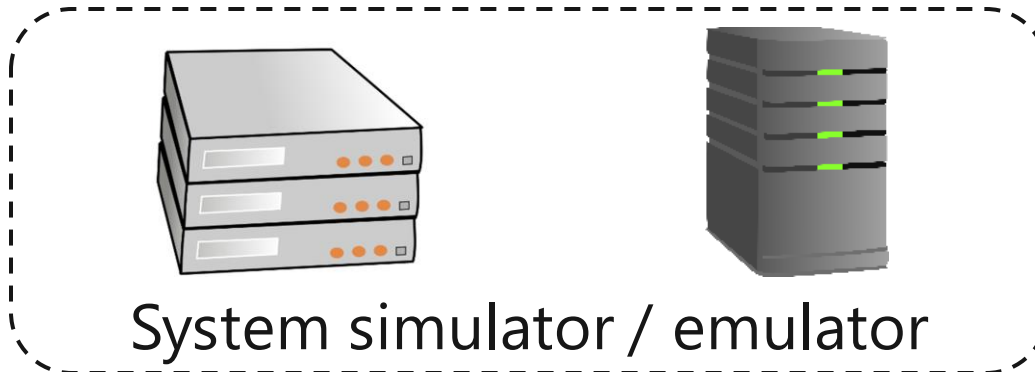


Reconfigure workloads & system

Evaluating an In-development System



No performance report

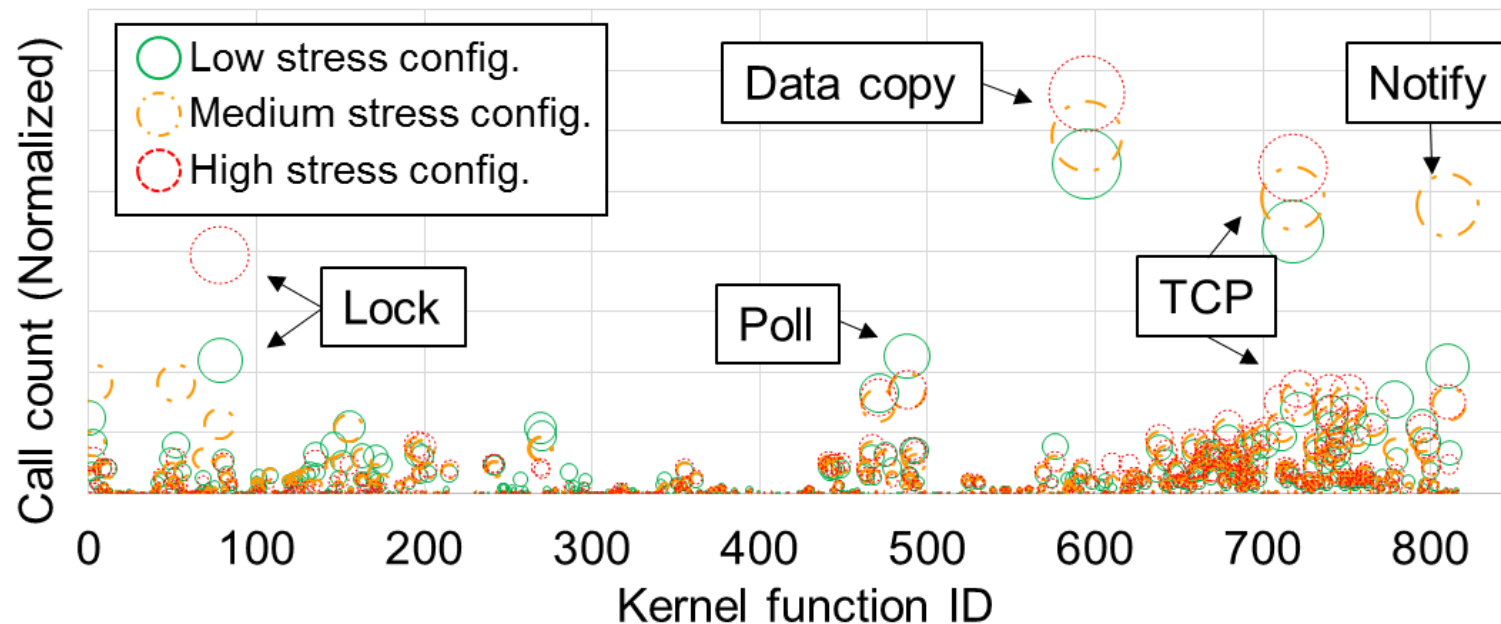


Investigate
uArchitecture details

System modeling tools: Too slow or too inaccurate

Workload Configuration Matters

- Δ Configuration $\rightarrow$ Δ System behavior
 - The system executes **different code patterns**
 - $\Rightarrow$ Different analysis results & system design insights



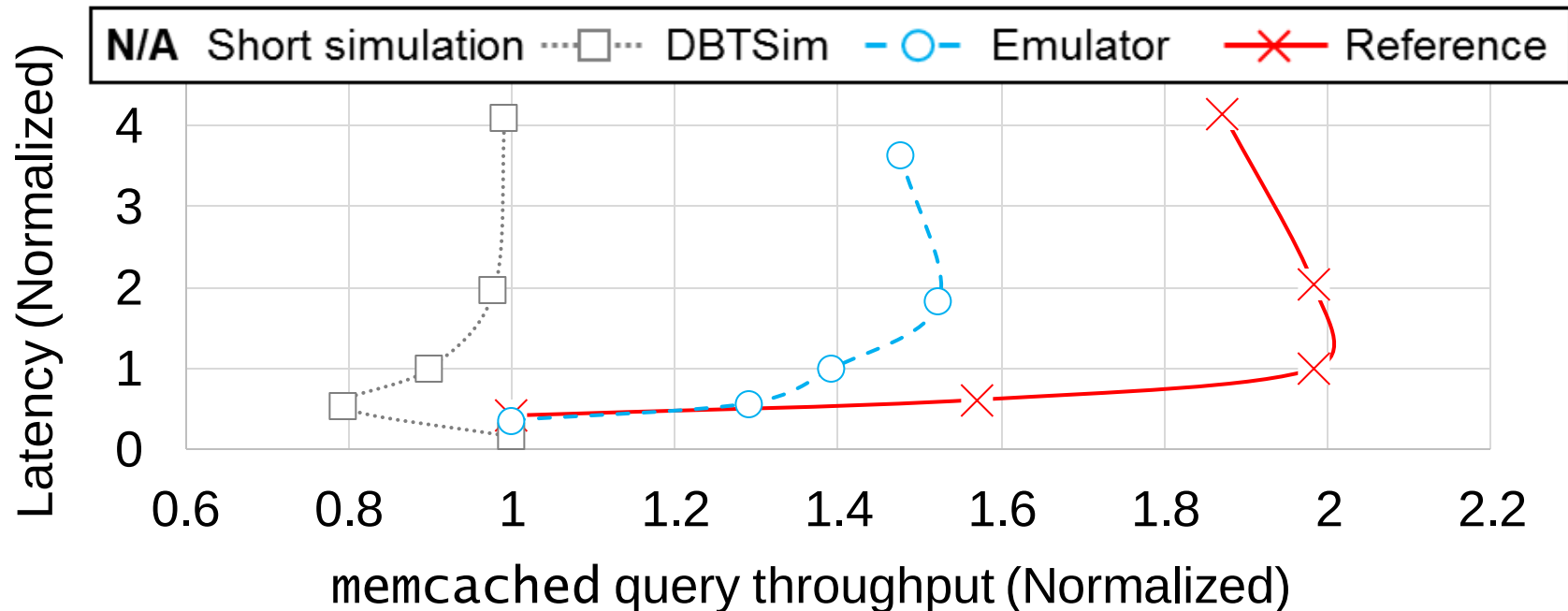
Must configure to represent actual usage scenarios

Index

- Introduction / Motivation
- Limitations
- Proposed idea: StressRight
- Evaluation
- Conclusion

Limitations (of the Existing Methods)

- **Inaccurate insights about the configurations**
 - Short simulation: **No high-level metrics**
 - DBT-based simulation: **No kernel considerations**
 - Emulator: **No timing considerations**



Index

- Introduction / Motivation
- Limitations
- **Proposed idea: StressRight**
 - Goals & Key ideas
 - Method details
- **Conclusion**

StressRight: Goals

- **Goals**

(e.g., latency, throughput)

- Quickly derive workload-reported performance metrics
 - To explore workload configurations for in-devel systems
 - To evaluate the systems with right stress behaviors

- **Requirements**

- Long workload execution
 - Must observe high-level workload-reported performance metrics
- Efficient performance model
 - To quickly derive the performance metrics

StressRight: Key Ideas

- **Long workload execution**

- Use timing-agnostic platforms (e.g., Emulators)

- ⇒ Extract **user & kernel** behavior, analyze performance later

- **Efficient performance model**

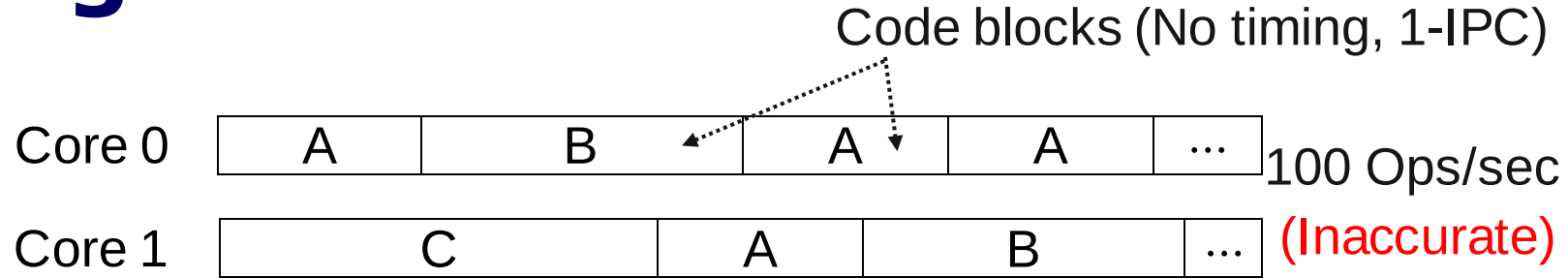
- Leverage redundancy in workloads

- ⇒ Analyze only the **unique behaviors** (i.e., code blocks)

- ⇒ **Overall behavior** = $\sum$ Analyzed unique behaviors

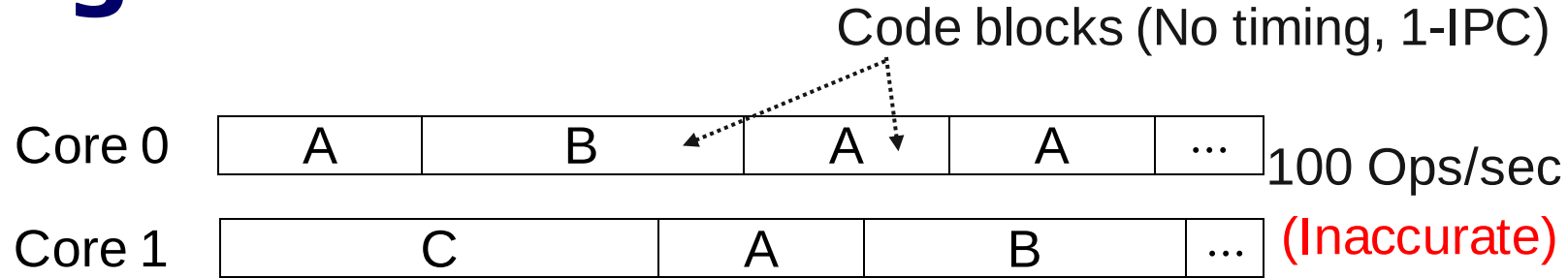
StressRight: Overview

Emulation

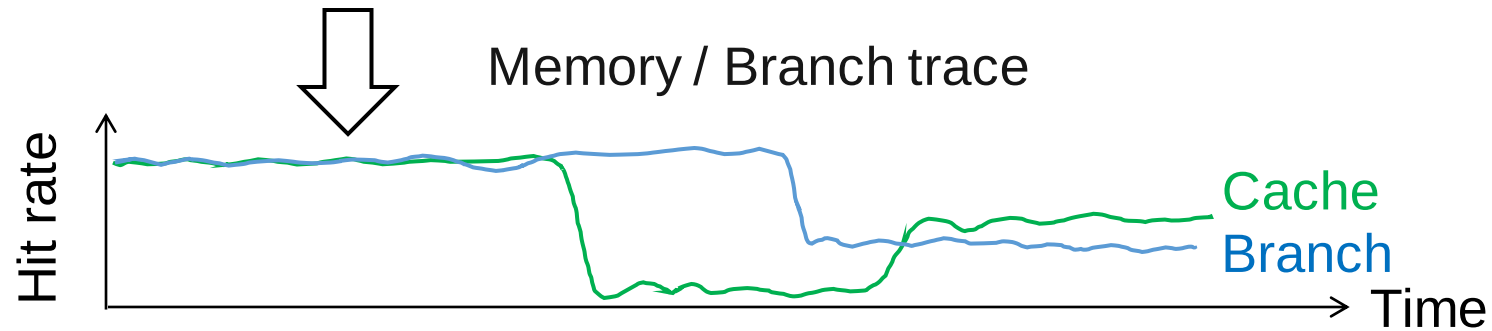


StressRight: Overview

Emulation

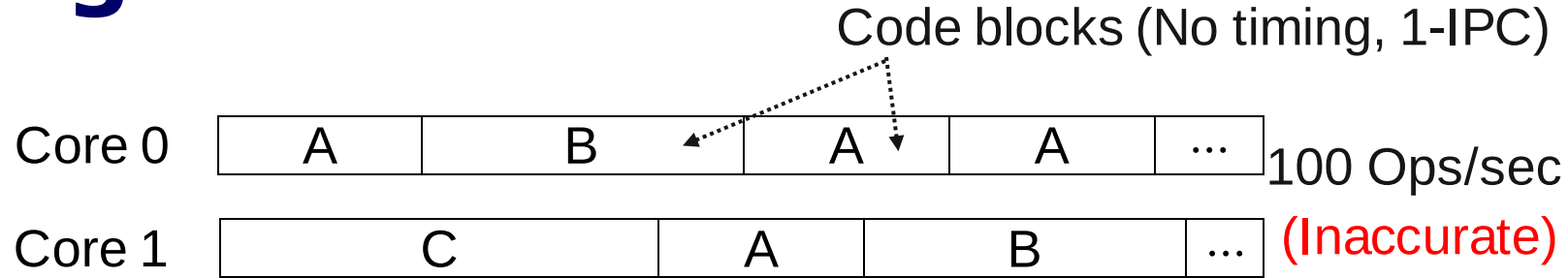


Functional simulation

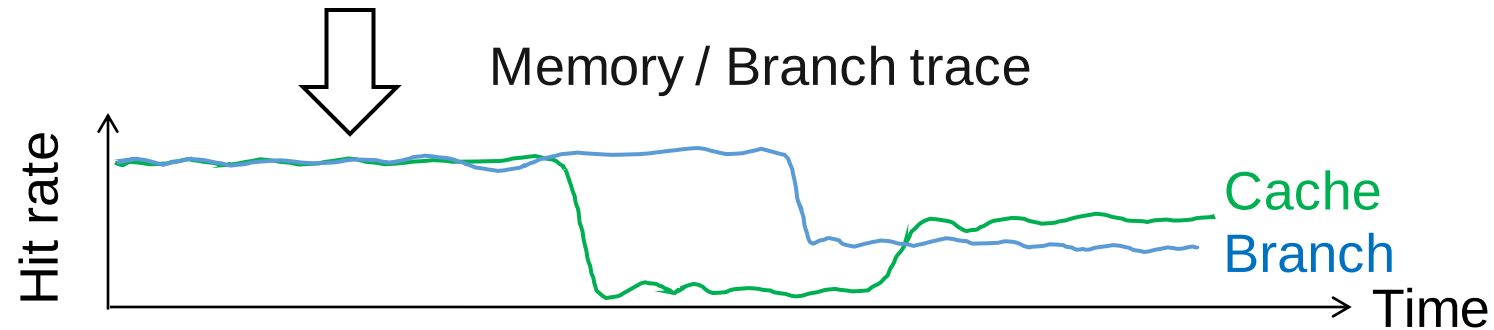


StressRight: Overview

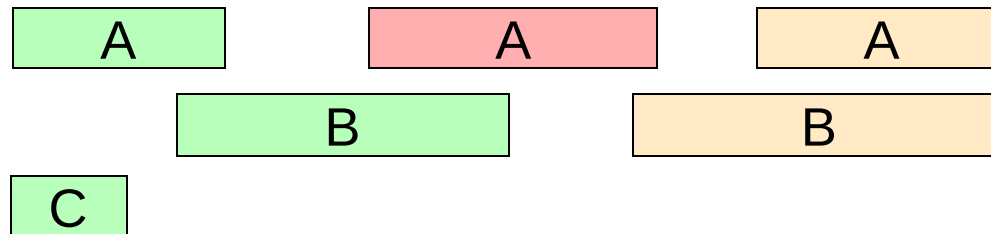
Emulation



Functional simulation

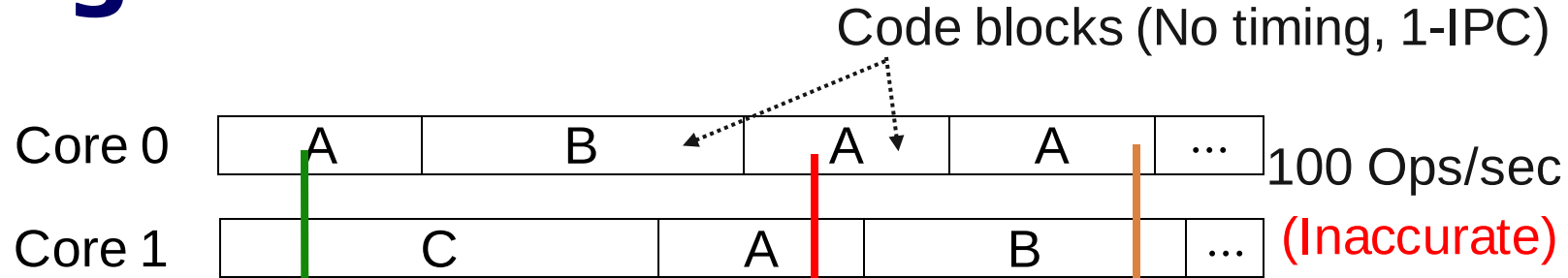


Timing reconstruction

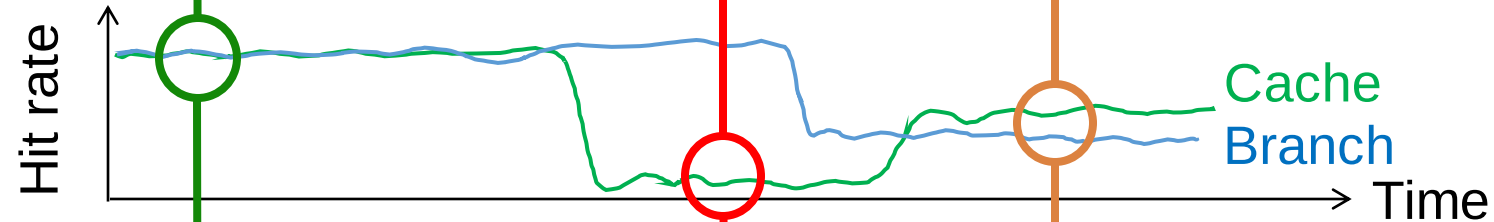


StressRight: Overview

Emulation



Functional simulation

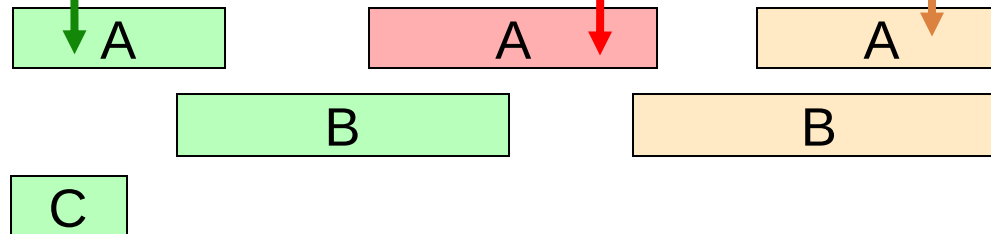


High \$ hit

Low \$ hit

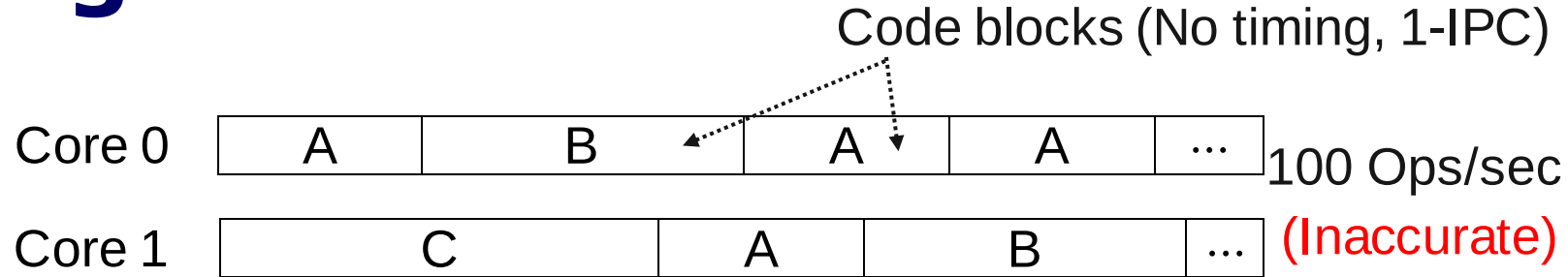
Med \$ hit

Timing reconstruction

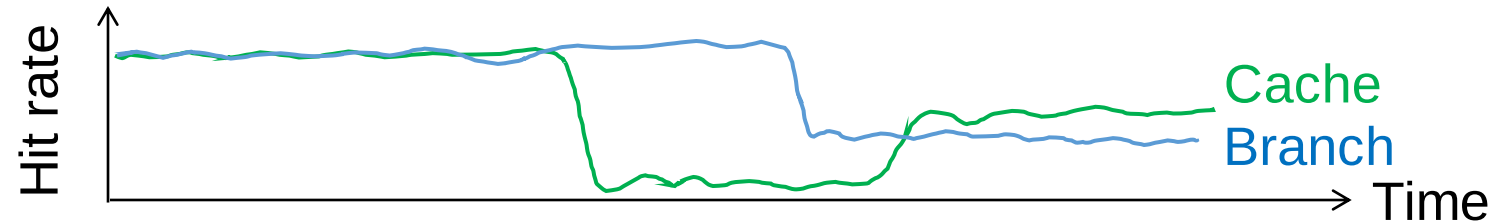


StressRight: Overview

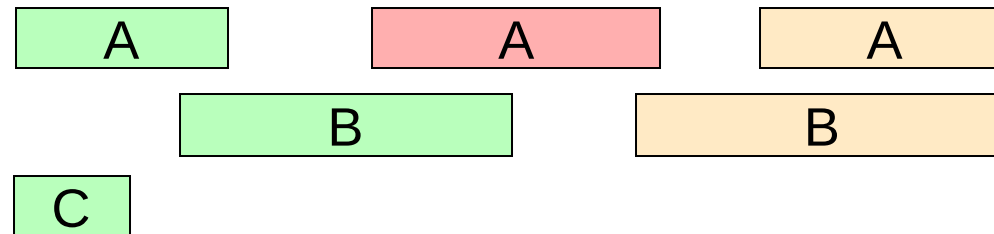
Emulation



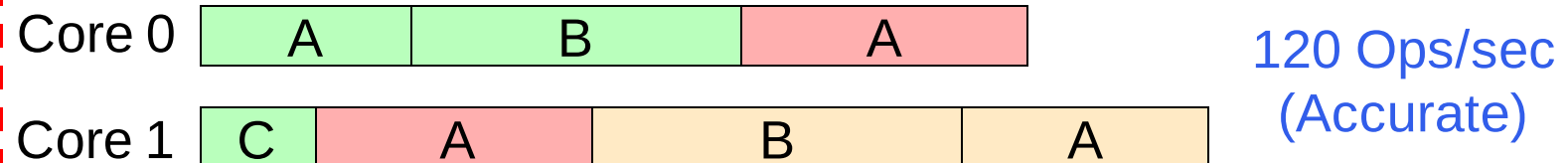
Functional simulation



Timing reconstruction

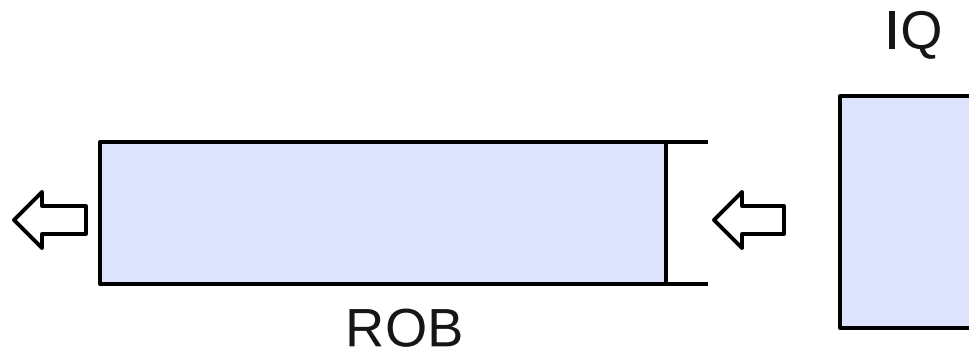


Reschedule & Reinterpret



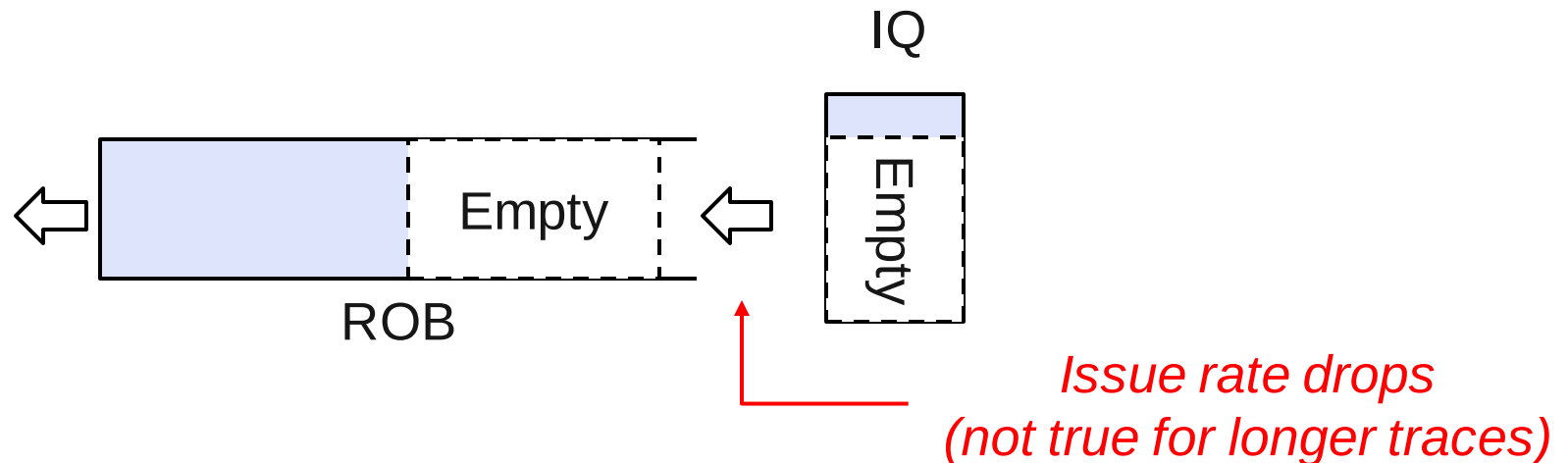
StressRight: Timing Reconstruction

- **Challenge: Code blocks are too short**
 - Pipeline drain effect is **nontrivial**



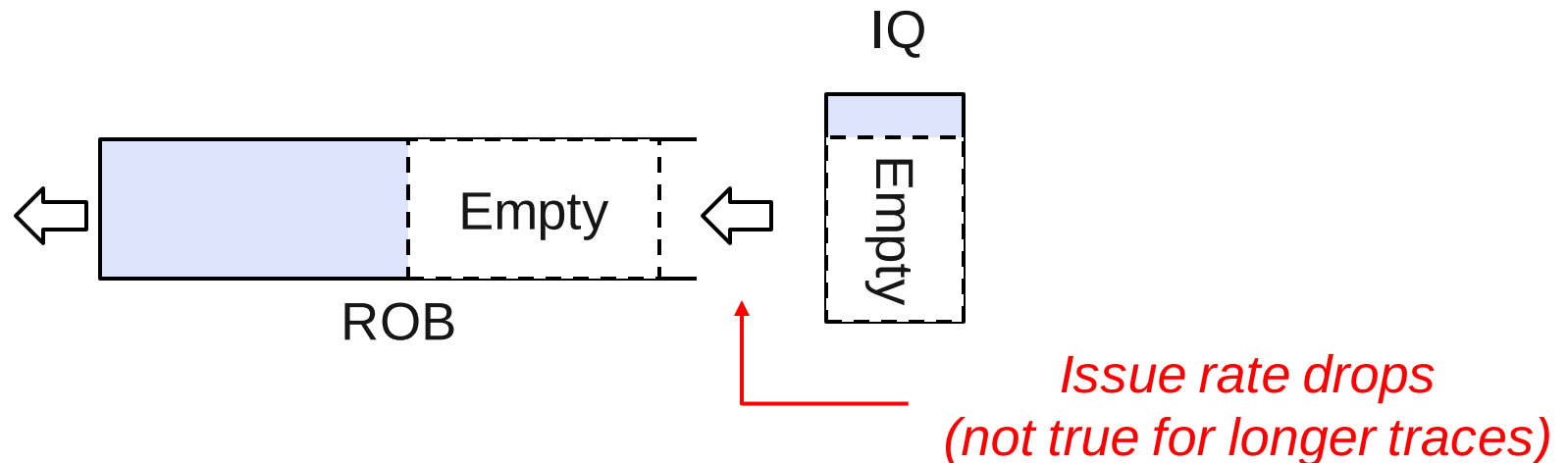
StressRight: Timing Reconstruction

- **Challenge: Code blocks are too short**
 - Pipeline drain effect is **nontrivial**



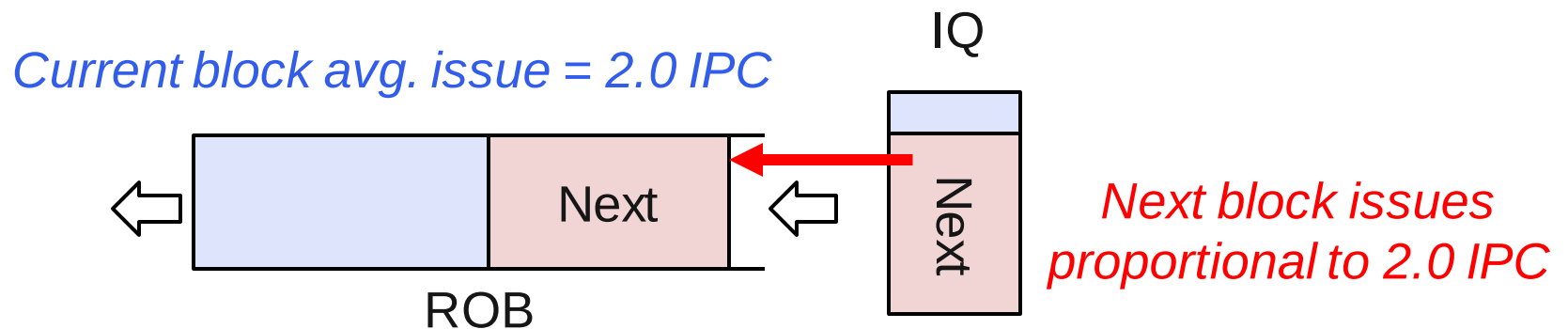
StressRight: Timing Reconstruction

- **Challenge: Code blocks are too short**
 - Pipeline drain effect is **nontrivial**
- **Solution: Consider hypothetical next block**
 - Assume: next block issue rate $\approx$ current block issue rate
 - Use power law* to further adjust the rate



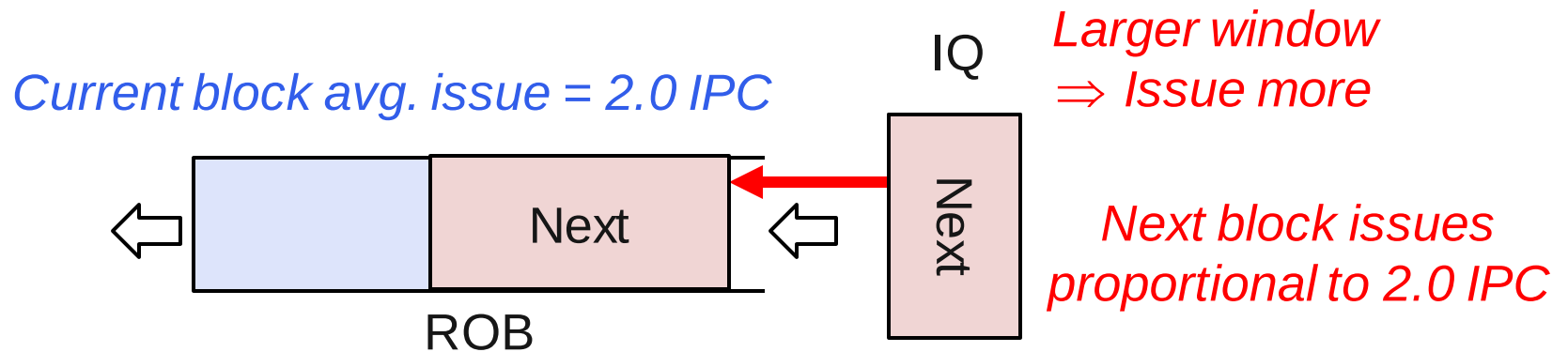
StressRight: Timing Reconstruction

- **Challenge: Code blocks are too short**
 - Pipeline drain effect is **nontrivial**
- **Solution: Consider hypothetical next block**
 - Assume: next block issue rate $\approx$ current block issue rate
 - Use power law* to further adjust the rate



StressRight: Timing Reconstruction

- **Challenge: Code blocks are too short**
 - Pipeline drain effect is **nontrivial**
- **Solution: Consider hypothetical next block**
 - Assume: next block issue rate $\approx$ current block issue rate
 - Use power law* to further adjust the rate



StressRight: Multiple Performances

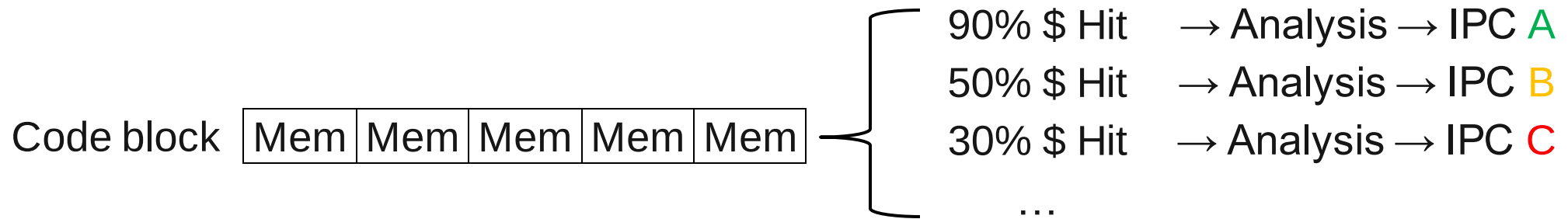
- **Challenge: Difficult to model every scenario**

Code block

Mem	Mem	Mem	Mem	Mem
-----	-----	-----	-----	-----

StressRight: Multiple Performances

- Challenge: Difficult to model every scenario



StressRight: Multiple Performances

- **Challenge: Difficult to model every scenario**
- **Solution: Mix template scenarios**
 - Random-generate scenarios & mix them
 - Few templates are enough

Code block

Mem	Mem	Mem	Mem	Mem
-----	-----	-----	-----	-----

60% Hit template

Hit	Miss	Hit	Hit	Miss
-----	------	-----	-----	------

IPC 2.0

40% Hit template

Hit	Miss	Miss	Hit	Miss
-----	------	------	-----	------

IPC 1.6

StressRight: Multiple Performances

- **Challenge: Difficult to model every scenario**
- **Solution: Mix template scenarios**
 - Random-generate scenarios & mix them
 - Few templates are enough

Code block

Mem	Mem	Mem	Mem	Mem
-----	-----	-----	-----	-----

60% Hit template	<table border="1"><tr><td>Hit</td><td>Miss</td><td>Hit</td><td>Hit</td><td>Miss</td></tr></table>	Hit	Miss	Hit	Hit	Miss	IPC 2.0
Hit	Miss	Hit	Hit	Miss			

40% Hit template	<table border="1"><tr><td>Hit</td><td>Miss</td><td>Miss</td><td>Hit</td><td>Miss</td></tr></table>	Hit	Miss	Miss	Hit	Miss	IPC 1.6
Hit	Miss	Miss	Hit	Miss			

50% Hit

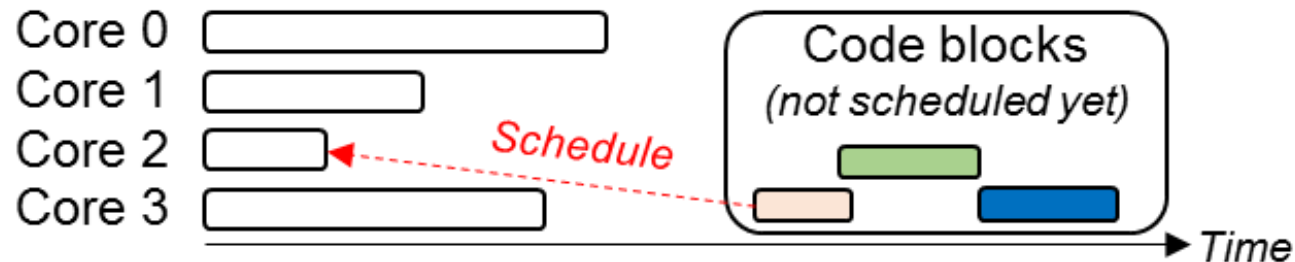
IPC 1.8

StressRight: Rescheduling

- **Basic scheduling method**

- Schedule to the earliest possible slot

Basic scheduling method

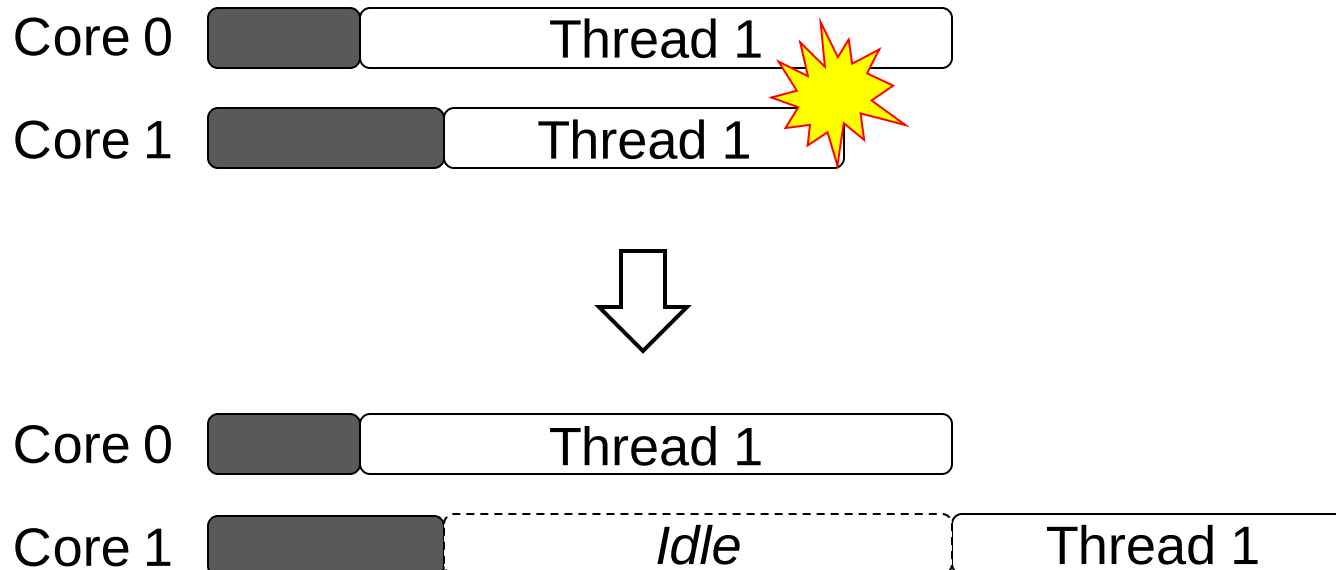


- **Three rules**

- Rule 1: Blocks from a thread execute serially
- Rule 2: Critical sections shouldn't overlap
- Rule 3: Threads should wait for barriers

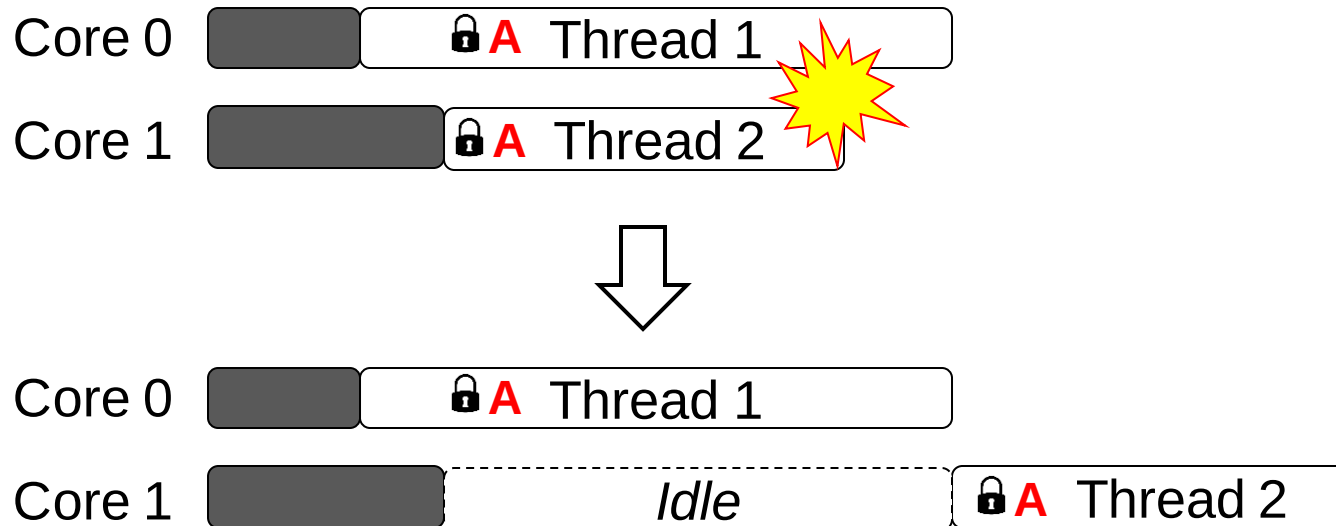
StressRight: Rescheduling

- **Rule 1: Blocks from a thread execute serially**
 - Tag code blocks with the **executor thread ID**
 - Prohibit blocks from a thread from running concurrently



StressRight: Rescheduling

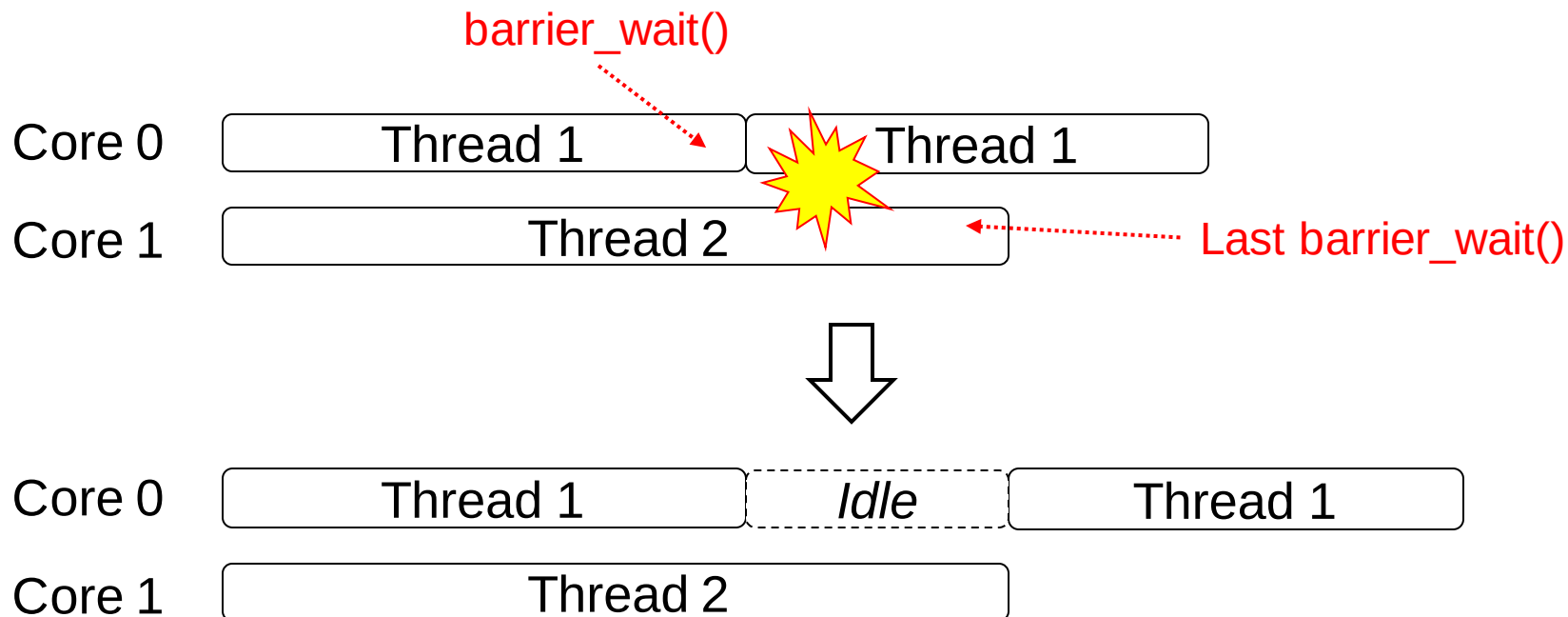
- **Rule 2: Critical sections shouldn't overlap**
 - Tag code blocks with **synchronization variable ID** (if applicable)
 - Prohibit the critical sections from overlapping



StressRight: Rescheduling

- **Rule 3: Threads should wait for barriers**

- Tag code blocks related to **barrier operations** (if applicable)
- Prohibit the scheduling before the last `barrier_wait()`



Index

- Introduction / Motivation
- Limitations
- Proposed idea: StressRight
- **Evaluation**
- **Conclusion**

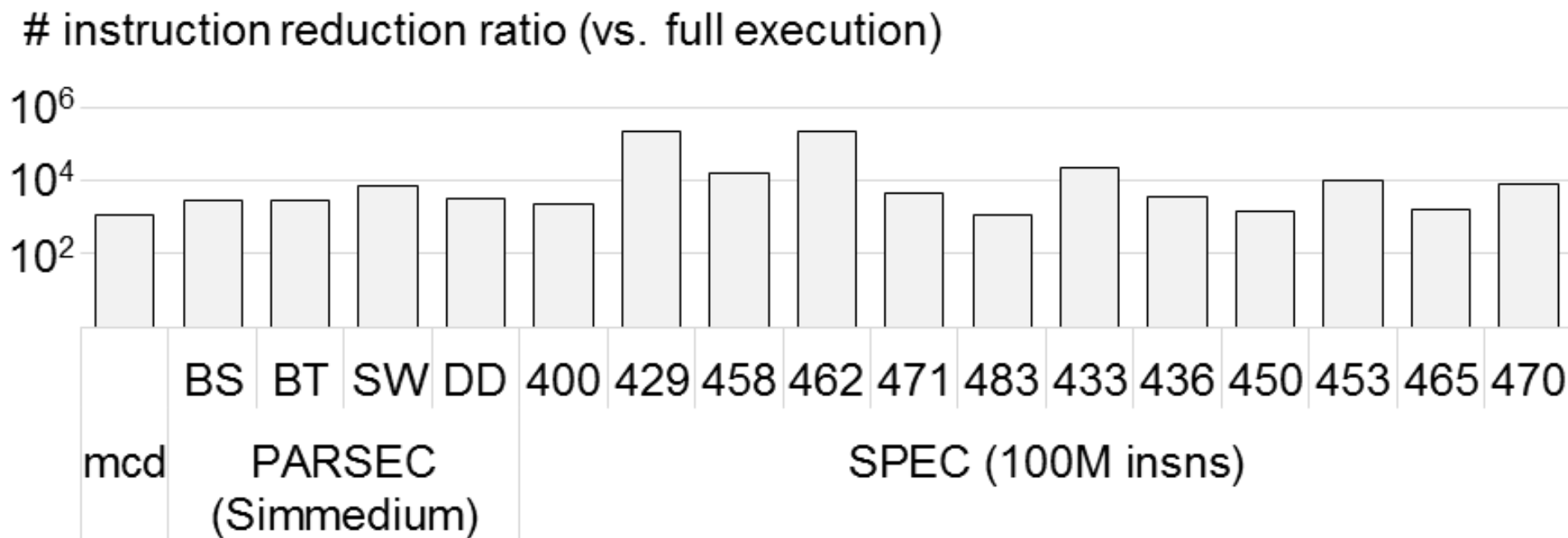
Evaluation

- **Quantitative analysis**
 - Why StressRight would work well
- **Accuracy and speed**
 - Comparison with cycle-level simulation (MARSSx86)
 - Model 1 / 12 / 16 OoO x86 cores
 - SPEC, PARSEC, memcached
- **Implementation**
 - Emulation: QEMU, Reconstruction models: Python, Functional simulators: C++

Quantitative Analysis

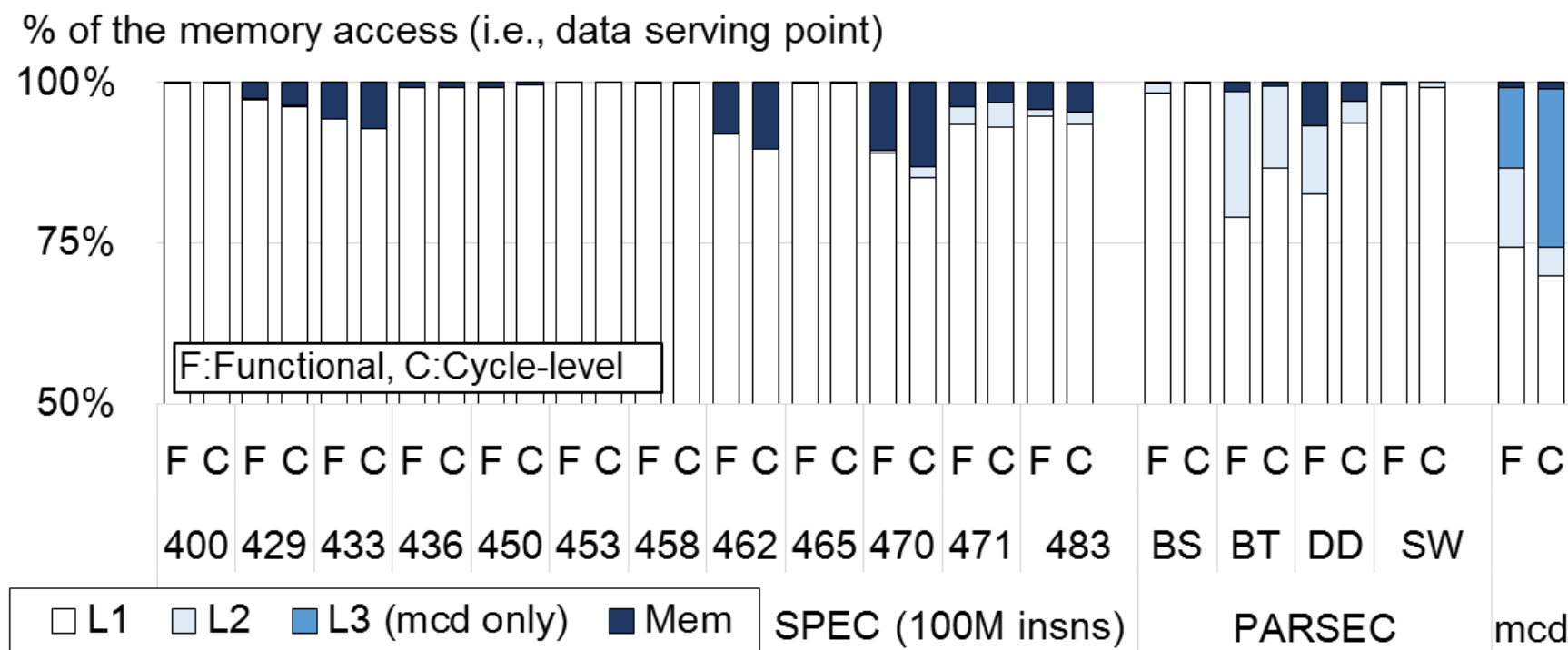
- **Efficiency of the method**

- # instructions: full-execution vs. unique code blocks
- **Orders of magnitude** reduction in the analysis load



Quantitative Analysis

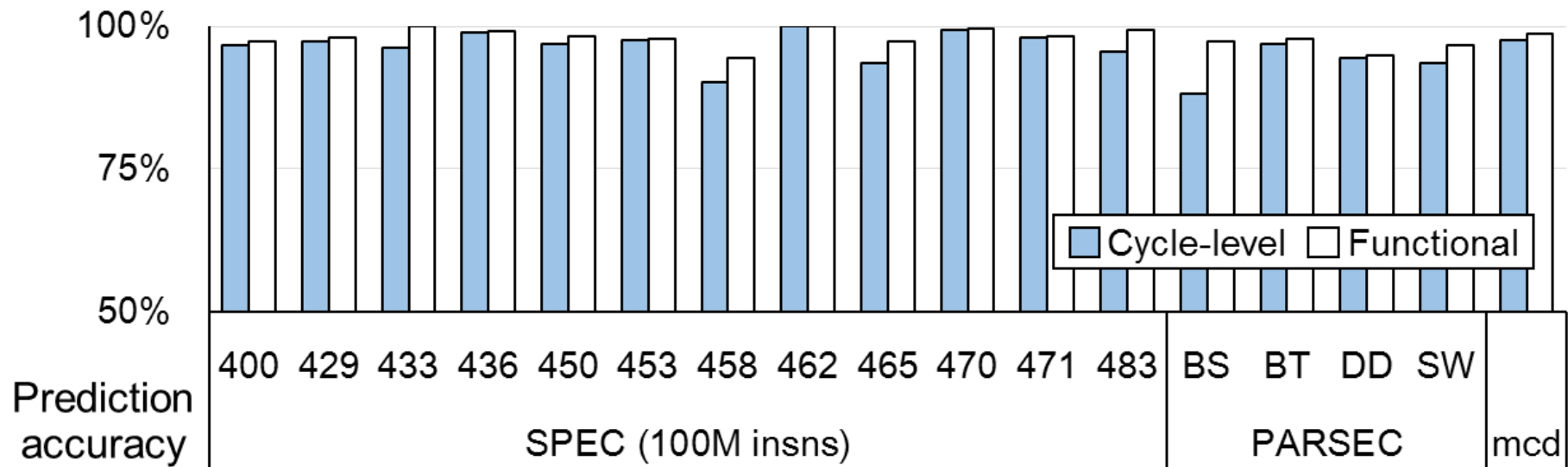
- **Accuracy of the dynamic resource models**
 - Functional simulations are **accurate enough**



Functional vs. Cycle-level memory simulation

Quantitative Analysis

- **Accuracy of the dynamic resource models**
 - Functional simulations are **accurate enough**

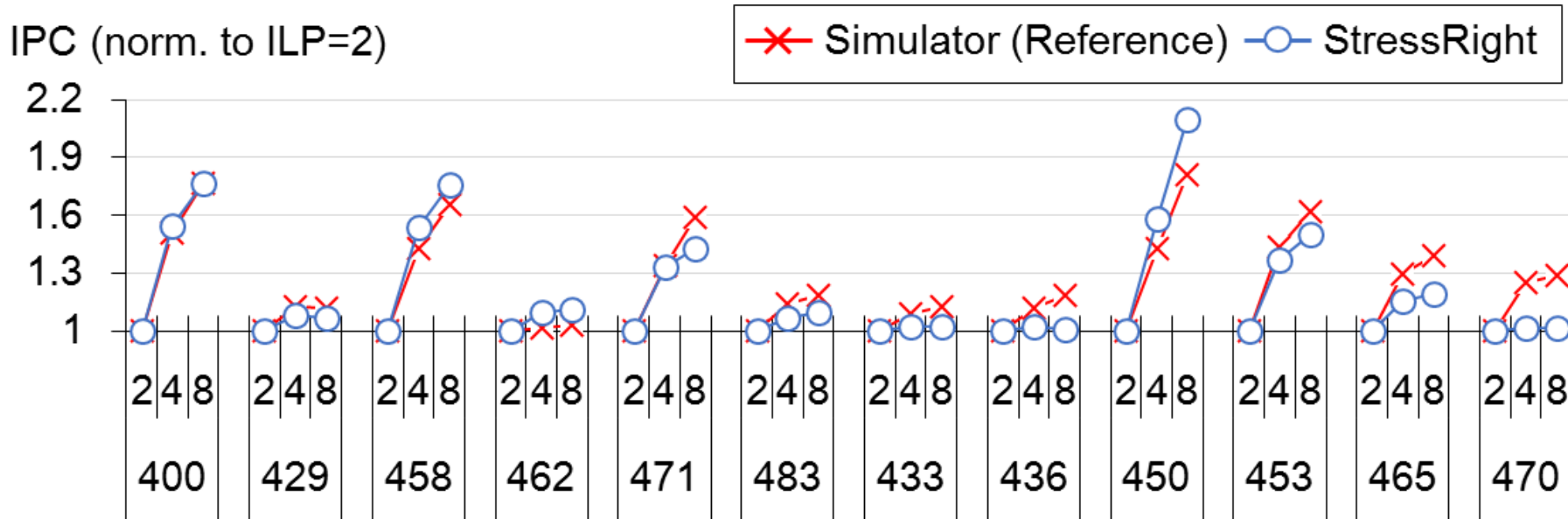


Functional vs. Cycle-level branch simulation

Accuracy: SPEC

- Validating the pipeline model

- Correctly estimates the first-order performance
 - Improvement in progress: Better memory model

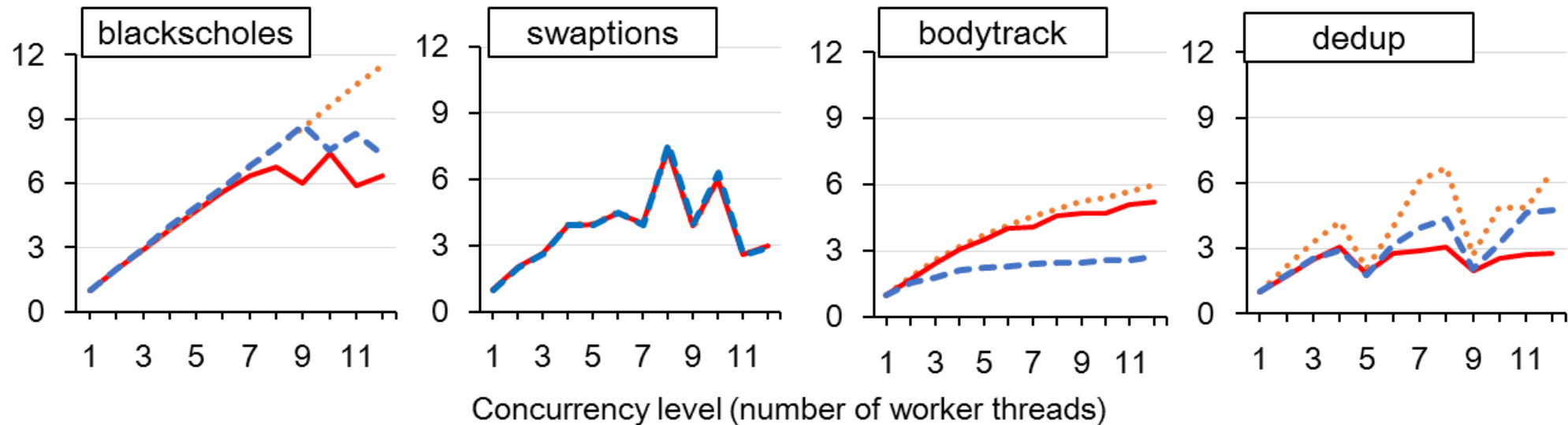


Accuracy: PARSEC

- **Validating the scheduler**

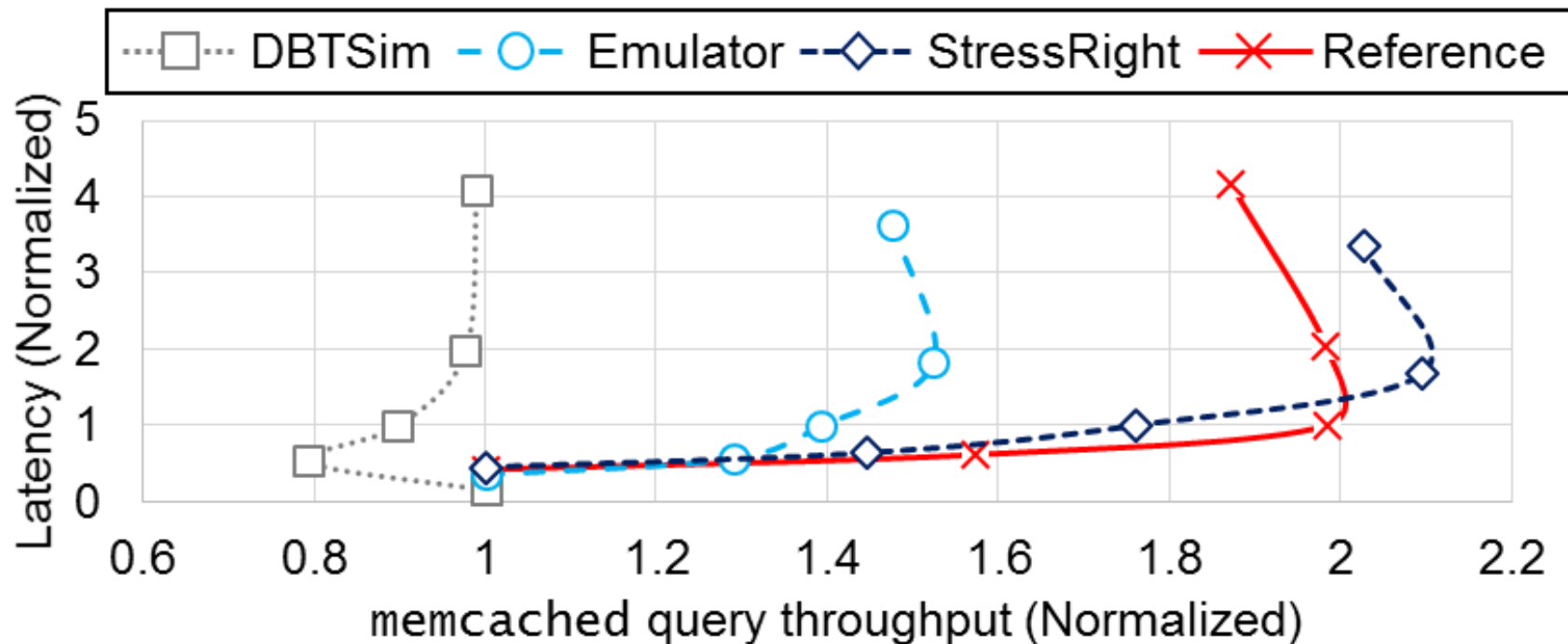
- **Correctly estimates** the scaling behavior
 - Improvement in progress: Barrier synchronizations

Speedup (vs. concurrency=1)



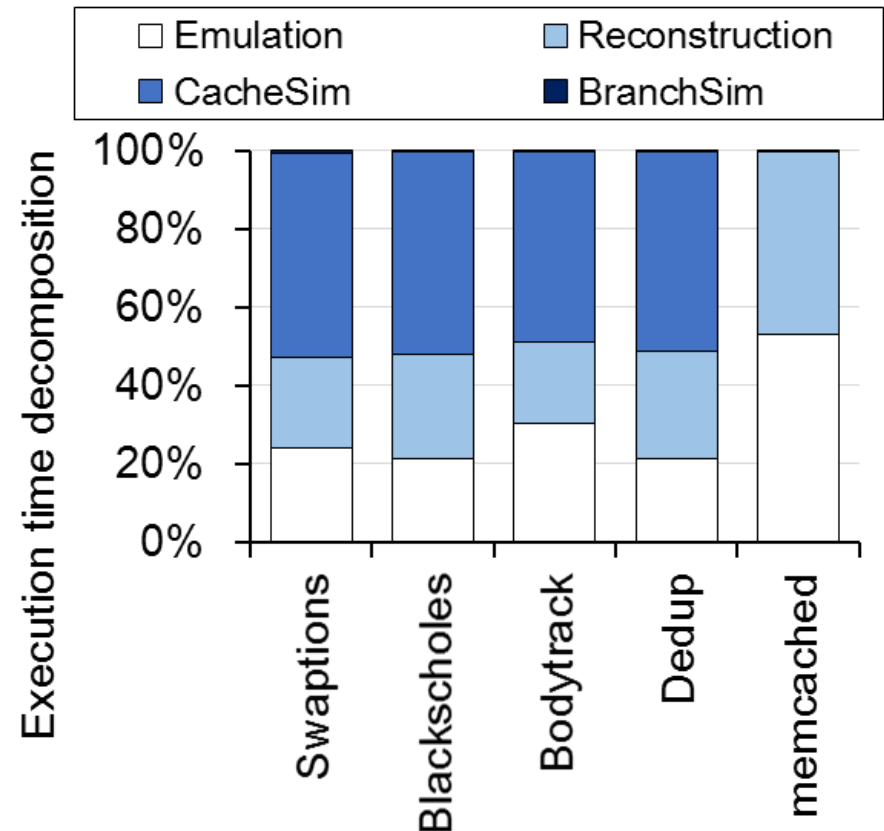
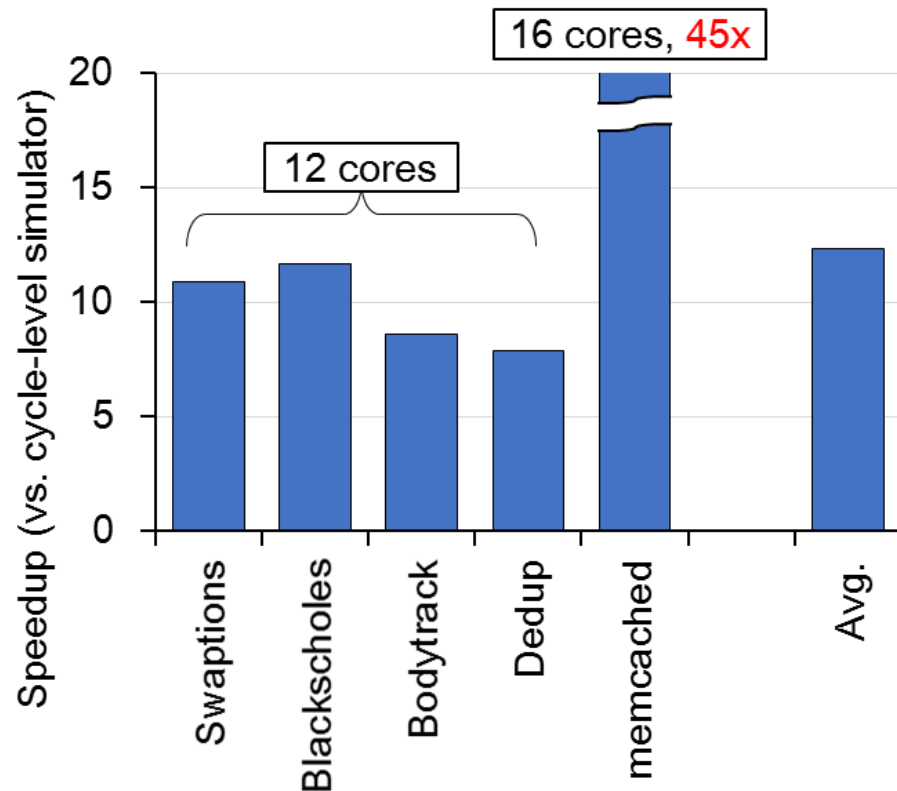
Accuracy: memcached

- **Reconstructing throughput-latency curve**
 - StressRight greatly improves over the existing methods



Speed evaluation

- Order of magnitude faster vs. simulator
 - Main bottleneck is cache simulation



*Reconstruction uses 40 vCPUs

Conclusion

- **Motivation**

- Stress in-development systems with **actual usage scenarios** to obtain **correct insights**

- **Key ideas**

- Focus only on unique behavior
- Consider execution dynamics: \$, branch, and scheduling

- **Results**

- **Accurately** reconstruct workload-reported performance metrics with an **order of magnitude** faster speed

Thank you!



Jaewon Lee (spiegel0@postech.ac.kr)

Pohang University of Science and Technology (POSTECH)